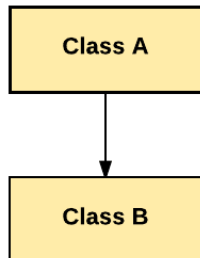


Inheritance (Theory)

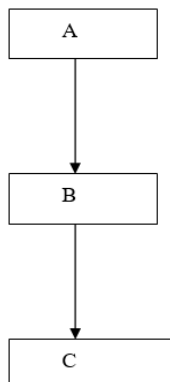
Inheritance is a feature of object-oriented programming where a class (child class, derived class, sub class) derives properties and behaviours (methods/functions) from another class (parent class, base class, super class), enabling code reuse and the creation of hierarchical relationships.

Types of inheritance:

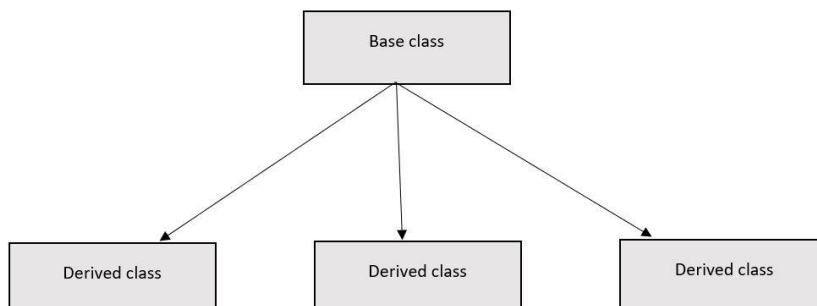
1. Single inheritance is a type of inheritance in which a child class inherits properties and methods from only one parent class. This allows the child class to reuse and extend the functionality of the parent class.



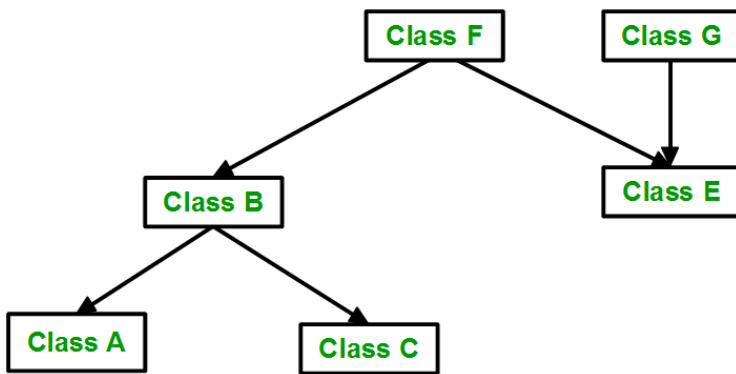
2. Multilevel inheritance is a type of inheritance in which a class (child class) inherits from another class (parent class), and that parent class itself is derived from another class (grandparent class). This forms a chain of inheritance, allowing properties and behaviours to pass through multiple levels.



3. Hierarchical inheritance is a type of inheritance in which multiple child classes inherit from a single parent class. Each child class can reuse and extend the properties and methods of the parent class independently.

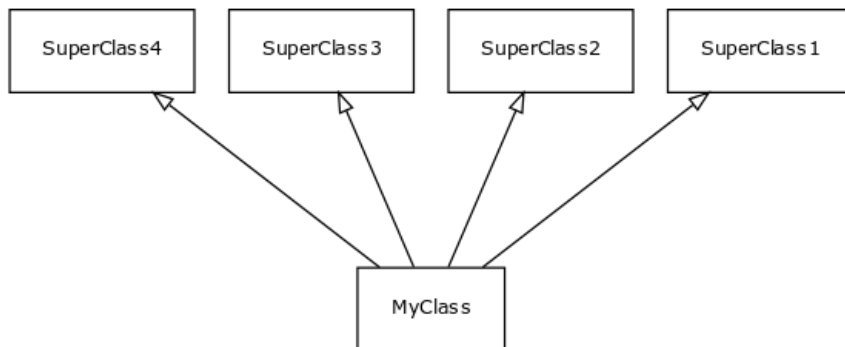


4. Hybrid inheritance is a combination of two or more types of inheritance, such as single, multilevel, and hierarchical inheritance. It allows a class to inherit properties and behaviors from multiple classes in different ways, forming a complex inheritance structure.



5. Multiple inheritance is a type of inheritance where a child class inherits properties and methods from more than one parent class. This allows the child class to combine and use the functionalities of all its parent classes.

****Java does not support multiple inheritance with classes to avoid ambiguity issues, such as the **diamond problem**. However, Java allows multiple inheritance through **interfaces**, which do not have method implementations.**



Access Specifiers:

1. public

- The member is accessible **from any class** in any package.
- Provides the **widest visibility**
- Public members of the parent class are fully inherited and accessible in the child class.

2. protected

- The member is accessible **within the same** and also **in subclasses** (even if they are in different packages).
- Provides visibility for inheritance

3. default (No Modifier)

- When no specifier is mentioned, it is **package-private** by default.
- The member is accessible **only within the same package**.
- Default (no modifier) members are inherited only if the child class is in the same package as the parent class.

4. private

- The member is accessible **only within the same class**.
- Provides the **most restricted visibility**
- Private members are **not inherited** by subclasses. However, subclasses can indirectly access them through public or protected methods in the parent class.

Variable types:

```
class Apple{  
    int a;                // instance variable  
    static int b;         // class variable  
    void sum(){  
        int k;           // local variable  
        if(true){  
            int p;        // local variable (block scoped)  
        }  
    }  
}
```

Scope:

1. Local Scope

- **Definition:** Variables declared inside a method, constructor, or block are called **local variables**.
- **Scope:** Accessible only within the method or block where they are declared.
- **Lifetime:** Created when the method or block is executed and destroyed once it is exited.
- **Default Value:** Local variables **do not have a default value**; they must be initialized before use.

2. Instance Scope

- **Definition:** Variables declared inside a class but outside of any method, constructor, or block, and without the static keyword, are called **instance variables**.
- **Scope:** Accessible by all methods of the class and specific to an instance of the class.
- **Lifetime:** Exist as long as the object of the class exists.
- **Default Value:** Initialized to default values (e.g., 0 for numbers, null for objects).

3. Static (Class) Scope

- **Definition:** Variables declared with the static keyword inside a class but outside of any method, constructor, or block are called **static variables**.
- **Scope:** Shared among all instances of the class and accessible using the class name.
- **Lifetime:** Exists for the entire duration of the program.
- **Default Value:** Initialized to default values (similar to instance variables)

4. Block Scope

- **Definition:** Variables declared inside a block of code (e.g., inside loops, if-statements) are limited to the block they are defined in.
- **Scope:** Accessible only within the block where they are declared.
- **Lifetime:** Exist only during the execution of the block.